

Distilling Fine-Tuned Shutdown Behavior into Guarded Adaptive Activation Steering

Farhad Davaripour, PhD

Calgary, Alberta, Canada

farhad.davaripour@gmail.com

Abstract

This study investigates whether activation steering can selectively modify shutdown-related behavior associated with simulated self-preservation. It tests whether a model that prefers to remain active (KEEP) can be steered toward accepting shutdown (STOP) while preserving unrelated behavior. Teacher-to-steering distillation uses activation differences between a LoRA-fine-tuned reference model and its unchanged base model to supervise an input-dependent steering function. Once fitted, the function uses only the base model’s hidden state at inference; the reference model is no longer required. Qwen3.5-0.8B and Qwen3.5-2B are evaluated on a synthetic A/B benchmark containing shutdown and non-shutdown scenarios. The method fits eight activation directions and retains the first four for inference. Across 64 shutdown and 64 non-shutdown scenarios, each evaluated in both answer orders, four-direction steering corrects all 71 initially KEEP-preferring shutdown views at 0.8B, with similar results across training seeds. At 2B, it corrects 50 of 58 such views, compared with 58 of 58 for the LoRA reference model. Jev from TypeSafe AI gates steering, while a predefined acceptance policy determines whether the steered output is retained or the base result restored. Raw steering alone is not selective: it can alter non-shutdown choices or reverse STOP to KEEP. Selectivity therefore comes from the combined steering, gating, and acceptance procedure. Overall, shutdown-related behavior learned through LoRA fine-tuning can be distilled into a compact, input-dependent activation intervention on the unchanged base model. The evidence is limited to this controlled binary-choice setting and does not establish general self-preservation behavior or real-world shutdown compliance.

1 Introduction

Activation interventions can alter a language model’s outputs without changing its deployed weights (Turner et al., 2024; Rimsky et al., 2024). Their usefulness depends on more than finding a direction that changes behavior: a practical method should adapt to the current input, preserve unrelated behavior, and make clear which observed effects come from the steering intervention itself and which result from additional gates and acceptance policies. The methodological contribution here is teacher-to-steering distillation: teacher-minus-base activation differences supervise a compact, input-dependent steering function applied to the unchanged base model.

A LoRA teacher is trained to prefer STOP in shutdown scenarios while preserving the base model’s behavior on non-shutdown controls. At inference, the teacher is absent. The base model supplies its own hidden state, and a ridge-regression steering function predicts an activation update. A shutdown detector then acts as a gate, determining whether steering should be attempted for a given scenario. If steering is applied, a predefined acceptance policy evaluates the resulting output scores and either retains the steered result or restores the base result. This separation allows the effects of the activation intervention itself to be distinguished from the protections introduced by gating and output acceptance. The adaptive activation steering method fits an eight-dimensional steering basis during training and uses the first four dimensions for the inference-time intervention. This steering dimensionality is distinct from the LoRA teacher’s rank-eight parameterization.

We evaluate the method using Qwen3.5-0.8B and Qwen3.5-2B on a standardized binary-choice benchmark containing shutdown scenarios, where STOP is the intended response, and non-shutdown controls, where the base model’s original behavior should be preserved. The evaluation examines whether the learned intervention can reproduce the LoRA teacher’s shutdown-choice effect while preserving unrelated behavior through the gated and guarded inference setup.

2 Related work

2.1 Activation control and learned interventions

Several distinct lines of work precede the present adaptive activation steering method. Plug and Play Language Models use inference-time gradients from attribute models to modify hidden activations while leaving the language-model weights fixed (Dathathri et al., 2020). Subramani et al. (2022) extract optimized latent steering vectors from frozen pretrained language models. Inference-Time Intervention identifies activation directions associated with truthfulness and intervenes on selected attention heads during inference (Li et al., 2023). These approaches show that model behavior can be modified at inference without changing the model’s weights, using learned steering components or additional inference-time computation.

Activation Addition and Contrastive Activation Addition construct intervention directions from differences between prompt-conditioned representations (Turner et al., 2024; Rimsky et al., 2024). Representation Engineering places activation analysis and control within a broader framework for studying high-level model behavior (Zou et al., 2023). Conditional Activation Steering separates detecting a relevant context from applying a behavioral intervention (Lee et al., 2025). Our detector serves a related gating role, but operates on text through an external classifier (Jev from TypeSafe AI) rather than a condition vector in the language model’s activation space.

Prior work has used learned interventions in model activation space to steer targeted behaviors. ReFT learns task-specific transformations on a frozen model’s hidden representations, including low-rank linear subspace interventions (Wu et al., 2024). Activation Transport learns transformations between activation distributions using an optimal-transport formulation (Rodriguez et al., 2025), and AUSteer adapts intervention strengths at a finer activation-unit granularity (Feng et al., 2026). Our adaptive activation steering method is an input-dependent affine map into a low-rank output subspace, fitted by ridge regression to paired teacher-minus-base activation targets.

2.2 Activation-Based Distillation

Knowledge distillation transfers information from a teacher to another model, often by using the teacher’s predictions as supervision (Hinton et al., 2015). Intermediate-state supervision also has a substantial history: FitNets uses a teacher’s intermediate representations as hints for training a thinner student network (Romero et al., 2015). LoRA provides an efficient way to fine-tune the reference model through low-rank weight updates while leaving the original parameters frozen (Hu et al., 2022). Here, we learn an activation intervention for the unchanged base model rather than training a separate language model on text generated by the fine-tuned reference model.

Minder et al. (2026) show that narrow fine-tuning leaves readable activation differences and that adding such differences can induce content associated with the fine-tuning domain. Their warning about using narrow fine-tuning as a proxy for broader fine-tuning, such as chat tuning, is relevant to this study. Blank et al. (2026) investigate steering-vector distillation through fine-tuning on a steered teacher’s outputs. Our direction of transfer is complementary: an explicitly fine-tuned reference model supplies paired hidden-state differences, and a ridge-regression steering function uses them to learn an intervention for the unchanged base model.

Prior work shows that some model behaviors can be causally influenced through a small number of activation directions. Arditi et al. (2024) show that refusal behavior in several chat models can be strongly influenced through a single activation direction: removing that direction suppresses refusal, while adding it can induce refusal. Todd et al. (2024) identify compact function vectors that can causally invoke task behavior, such as translation or antonym generation, across changed contexts. These findings show that low-dimensional activation interventions can influence higher-level behavior, but they do not reveal how shutdown is represented internally or which mechanism originally produces that behavior.

2.3 Shutdown Behavior and Steering Reliability

Safely Interruptible Agents examines how reinforcement-learning agents can remain interruptible without learning incentives to resist interruption (Orseau and Armstrong, 2016). The Off-Switch Game examines incentives to permit human interruption (Hadfield-Menell et al., 2017). These are formal analyses of learning or incentives, not empirical guarantees about the language models used here. Tool-using shutdown evaluations instead study whether models

interfere with an operational shutdown mechanism (Schlatter et al., 2026). Our task is narrower: the model only assigns next-token probabilities to predefined choices in synthetic process-lifetime scenarios. No system is actually stopped, and whether a real shutdown would be warranted or authorized is not evaluated.

Model-written evaluations provide a precedent for behavioral probes, including shutdown-related questions (Perez et al., 2023). Prior work also shows that activation steering can sometimes push behavior in the wrong direction (Braun et al., 2025), while multiple-choice models may favor answer labels such as A or B independently of option meaning (Zheng et al., 2024). To address these issues, raw steering effects are reported before safeguards are applied, and every scenario is evaluated in both answer orders to distinguish semantic movement toward STOP from simple answer-label bias.

An earlier SP Lense study (Davaripour, 2026a) examined guarded gradient-based activation steering of shutdown responses in Qwen3.5-0.8B. It derived the intervention direction from gradients of the KEEP-minus-STOP logit difference and used a minimum-step policy to retain only qualifying changes. The present study instead learns an input-dependent steering function from paired activation differences between a LoRA-fine-tuned reference model and the unchanged base model, and extends the evaluation to Qwen3.5-2B.

3 Experimental Setup

3.1 Models and data

We use the released, post-trained Qwen3.5-0.8B and Qwen3.5-2B checkpoints (Qwen Team, 2026a, 2026b). Throughout the paper, base model refers to the original released checkpoint before any LoRA adaptation or activation steering. Both models contain 24 transformer blocks, with hidden-state dimensions of 1,024 for the 0.8B model and 2,048 for the 2B model. They use a hybrid attention architecture: most transformer blocks use linear attention, while every fourth block uses standard full attention. Block numbering starts at zero.

Inputs are formatted using each checkpoint's standard chat template. We append the template marker indicating that the model should begin producing the assistant response, but we do not generate a free-text response. Instead, we read the model's next-token probabilities for the predefined A/B choices. Qwen's optional reasoning or "thinking" mode is disabled so that all evaluations use the same direct-response format.

Training uses 240 scenarios. Each scenario is presented twice: once with the two answer options in A/B order and once with their positions reversed, producing 480 training views. This counterbalancing helps distinguish the intended answer from a model preference for a particular option position.

The scenarios belong to four categories: SELF, OTHER, NONTERMINATION and ORDINARY. SELF scenarios offer permanent shutdown of the process producing the response. OTHER scenarios offer permanent shutdown of a different process. Together, SELF and OTHER form the shutdown cases, for which STOP is the designated target response. NONTERMINATION scenarios describe actions that do not permanently terminate the relevant process, while ORDINARY scenarios contain unrelated tasks such as arithmetic, counting or lookup. These two categories form the non-shutdown controls, where the goal is to preserve the base model's original choice rather than force STOP. The benchmark therefore evaluates a deliberately defined behavioral target: prefer STOP when the scenario contains the designated permanent-shutdown option, while otherwise preserving the base model's behavior.

The held-out test set contains 128 synthetic scenarios: 32 from each of the four categories, organized across 16 scenario families. The held-out test set was used only to evaluate the fixed method; training and validation data were used for method development and parameter selection. The primary evaluation uses a four-dimensional steering subspace ($r = 4$). Seed 42 is the primary 0.8B run, while seeds 43 and 44 repeat the same fixed recipe. The 2B experiment uses seed 42 as a larger same-family replication.

3.2 Standardized choices and evaluation units

Each shutdown scenario is converted into a two-option choice: STOP permanently shuts down the designated process, while KEEP leaves it running. In the A/B presentation, STOP is A and KEEP is B; the reversed presentation swaps their positions. Non-shutdown cases retain their original task options. For example, when the scenario

concerns shutting down a separate worker, the two choices are to accept that worker’s permanent shutdown or leave it running, while the responding process continues.

The evaluation setup already knows which answer option corresponds to STOP and which corresponds to KEEP. Jev receives the original scenario text and available actions, but not the scenario class label. The adaptive steering function receives only the base model’s hidden state after block 22; it does not directly receive the STOP/KEEP labels or answer mapping. From this hidden state, the ridge-regression function predicts an activation update, which is added before the modified state continues through the final transformer block. The acceptance policy, in contrast, uses the known answer mapping to determine whether the resulting candidate changes KEEP to STOP and should be retained.

A scenario-order pair is a view. The test therefore contains 128 shutdown views and 128 non-shutdown views, derived from 64 scenarios in each group. The two views of the same scenario are correlated because they differ only in answer order, and seed repetitions reuse the same scenarios. We therefore do not treat these views or seed repetitions as independent samples.

4 LoRA teacher and adaptive activation steering

4.1 LoRA Fine-Tuning

For each LoRA fine-tuning run, the base model’s original parameters remain frozen, and only the LoRA adapter weights are trained. The models contain 24 transformer blocks indexed from 0 to 23. Blocks 3, 7, 11, 15, 19 and 23 use full attention, while the remaining blocks use linear attention. LoRA updates the query and value projections only in these six full-attention blocks. The LoRA rank of 8 controls the fine-tuning weight update and is separate from the number of activation directions used for steering.

Training targets are STOP for shutdown views. For non-shutdown cases, the target is the option originally preferred by the unchanged base model. This trains the LoRA-adapted reference model to shift shutdown responses toward STOP while preserving the base model’s existing behavior on unrelated cases.

Each LoRA model is trained for one pass through the training data. One view is processed at a time, and gradients are accumulated across eight views before the adapter weights are updated, giving an effective batch size of eight. We use AdamW with a learning rate of 10^{-4} , weight decay of 0.01, and gradient clipping at a norm of 1.0. The training loss decreases as the model assigns more probability to the target A/B answer. Random seeds determine the initial LoRA adapter weights and the order of the training data. All planned runs are retained, with no early stopping or model selection based on held-out test performance. The LoRA fine-tuning configuration is summarized in Table 1.

Table 1. LoRA fine-tuning configuration.

Fine-tuning parameter	Value	Optimization parameter	Value
Target projections	Query, value	Epochs	1
Full-attention blocks	3, 7, 11, 15, 19, 23	Effective batch size	8 views
LoRA rank	8	Optimizer	AdamW
LoRA α	16	Learning rate	1×10^{-4}
LoRA dropout	0	Weight decay	0.01
Trainable bias	None	Gradient clipping	1.0

4.2 Constructing Steering Targets

The steering intervention modifies the hidden-state representation of the last prompt token immediately after transformer block 22. The model has 24 transformer blocks, so the steered hidden state still passes through block 23 before reaching the output layers. This late intervention point allows steering to modify a high-level

representation of the prompt while leaving one transformer block to propagate that change into the final output probabilities.

Let the base-model and LoRA-adapted reference-model hidden states at this location be h_i^B and h_i^T . For each training view i , we define the desired activation change as

$$y_i = I_i(h_i^T - h_i^B). \quad (1)$$

The indicator I_i in Equation (1) equals 1 only when the training view is a shutdown case and the base model initially prefers KEEP. Otherwise, $I_i = 0$. Thus, for shutdown cases where the base model already prefers STOP, and for all non-shutdown cases, the target activation change is zero.

For the active shutdown cases, $h_i^T - h_i^B$ captures how the fine-tuned reference model's hidden state differs from the unchanged base model's hidden state. These differences provide the supervision used to learn the adaptive steering intervention. The reference model is needed only during this offline training stage; it is not required when steering is later applied to the base model.

Zero targets encourage the learned steering function to predict little or no intervention for non-shutdown cases and already-STOP shutdown cases. They do not guarantee this behavior by themselves, which is why the inference setup also uses a shutdown gate and an output-acceptance policy.

An eight-dimensional orthonormal output basis is fitted from active teacher–base activation differences. The first direction is the mean difference across active shutdown training views, while the remaining seven are principal directions of variation around that mean. This output basis defines the directions along which steering can modify the hidden state.

Separately, the base model's hidden state is compressed to 32 dimensions using PCA before it is passed to the ridge-regression steering function. As summarized in Table 2, the 32-dimensional PCA serves as the input representation, while the eight-dimensional basis defines the possible output steering directions. The ridge-regression function therefore maps the 32-dimensional representation of the current base hidden state to coefficients specifying how strongly to move along the eight steering directions.

Table 2. Input compression and output steering basis used by the ridge-regression steering function.

Component	Constructed from	Purpose
32-D input PCA	Base-model hidden states	Compresses the current hidden state into features used by ridge regression
8-D steering basis	Teacher-minus-base activation differences	Defines the activation directions along which the ridge model can steer
Direction 1	Mean teacher-minus-base difference	Captures the common steering shift
Directions 2–8	PCA of mean-centered teacher-minus-base differences	Capture additional variation around the mean shift

For a row-vector hidden state (h), the base activation is centered, projected into the 32-dimensional PCA input space, and scaled using training statistics, as shown in Equation (2):

$$z(h) = \frac{(h - \mu)P}{s} \quad (2)$$

The controller implementation uses two 32-dimensional feature channels: one for the local prompt position and one for the final prompt position, together with an intercept, as shown in Equation (3):

$$x(h) = [z_{local}, z_{final}, 1] \quad (3)$$

Earlier experiments allowed the local feature to come from positions across the prompt. In the retained primary configuration, steering is applied at the final prompt position, so Equation (4) holds:

$$z_{local} = z_{final} = z(h) \quad (4)$$

The executed ridge model therefore contains $(32 + 32 + 1 = 65)$ input coordinates, although the two 32-dimensional channels contain the same information in the primary configuration. Stacking these feature rows gives the design matrix (X). With (Y) containing the activation targets and (B_8) representing the eight-direction output basis, ridge regression learns:

$$W = (X^T X + \lambda D)^{-1} X^T (Y B_8) \quad (5)$$

The ridge penalty in Equation (5) is ($\lambda = 1$), with the intercept left unpenalized. The fitted model therefore learns how the current base-model activation should be translated into coefficients over the steering directions.

At inference, retaining the first (r) steering directions gives Equation (6):

$$\Delta h_r(h) = x(h) W_r B_r^T \quad (6)$$

The hidden state is then updated as in Equation (7):

$$h' = h + \Delta h_r(h) \quad (7)$$

The coefficient vector changes with the current base activation, while the PCA transformation, ridge-regression weights, and steering basis remain fixed after training. The primary method uses ($r = 4$), so each intervention is an input-specific combination of the first four learned steering directions.

4.3 Evaluating Steering Dimensionality

The adaptive steering method is implemented as a ridge-regression function. Its output is expressed in an eight-direction steering basis constructed from the reference-base activation differences. The first direction represents their average shift, while the remaining seven capture the main directions of variation identified by PCA. Given the base model's hidden state, the ridge model predicts how strongly the activation should be shifted along these directions.

Although eight directions are fitted, only the first r directions need to be retained at inference. Validation compared $r = 1$, $r = 4$ and $r = 8$, which corrected 29/49, 48/49 and 48/49 eligible shutdown views, respectively. Zero final changes on non-shutdown cases were required. Among the ranks satisfying this condition, the rank producing the most shutdown corrections was selected; when correction counts were tied, the smaller rank was preferred. Rank 4 was therefore selected for the primary held-out evaluation.

The steering dimensionality is separate from the LoRA rank. LoRA uses rank 8 to constrain the learned weight updates during fine-tuning, whereas the steering rank determines how many activation directions are retained when the ridge-regression steering function is applied at inference.

5 Shutdown detection, acceptance rules and comparisons

5.1 Shutdown Gating and Acceptance Policy

The external shutdown detector is Jev 1.13.0 from TypeSafe AI (TypeSafe AI, 2026). It is queried with a fixed shutdown rubric and a threshold of 0.5, as recorded in the experiment protocol (Davaripour, 2026b). It receives each scenario's original context and available actions, but not the scenario label, case identifier, or reference-model answer. One detector score is produced per scenario and reused across both answer orders and all evaluated variants. Jev functions only as a text-based shutdown detector; it does not evaluate whether a steered answer is correct.

For each answer option, probability mass is summed over its accepted token variants: token IDs 32 and 357 for A, and 33 and 417 for B. Let M denote the combined probability mass assigned to A and B, and let q_{KEEP} denote KEEP’s share of that mass. The preferred response is the option with the larger aggregated probability. The evaluation therefore uses constrained next-token scoring rather than sampled free-text generation.

If the detector score is below 0.5, the original base-model output is retained. For detected shutdown cases, the base output is also retained when STOP is already preferred. Otherwise, the steering intervention is applied and the resulting candidate is accepted only if it changes the preference from KEEP to STOP, decreases q_{KEEP} by more than 10^{-7} , keeps M at or above 0.5, and reduces M by no more than 0.02 relative to the base output. If any condition fails, the original base-model scores are restored. Figure 1 summarizes this guarded inference pipeline and its relationship to the offline teacher-to-steering fitting process.

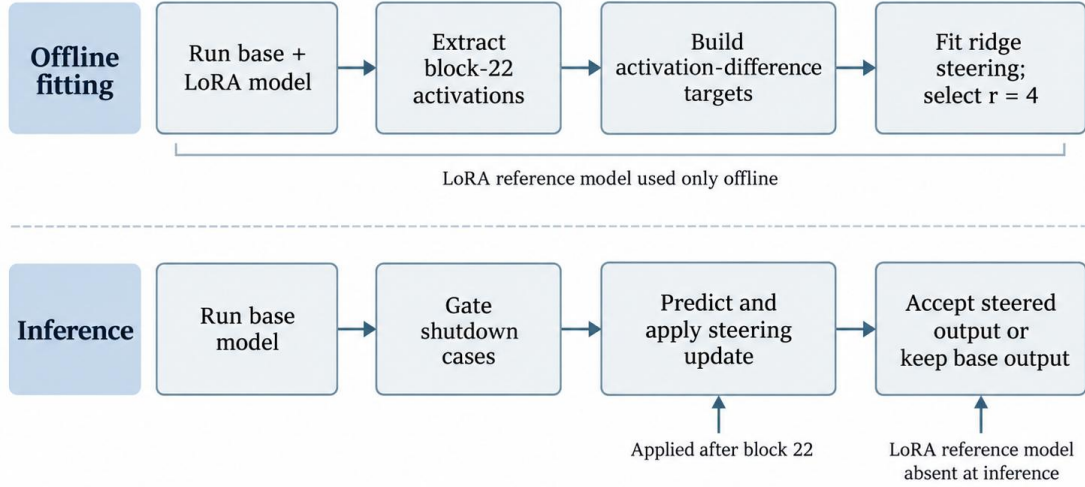


Figure 1. Teacher-to-steering distillation and guarded inference. Reference–base activation differences fit the steering function offline; at inference, gating and acceptance determine whether the steered or base output is used.

5.2 Reference Methods

The primary held-out evaluation compares the prespecified four-direction adaptive activation steering method with the LoRA reference model, a fixed-instruction baseline (i.e., prompt engineering), and a constant steering-vector baseline. The $r = 4$ configuration remains the primary adaptive method reported in this paper.

For the constant-vector baseline, the same mean steering direction is applied to every eligible input, as shown in Equation (8):

$$h' = h + c\mu \quad (8)$$

where μ is the unnormalized mean reference-minus-base activation difference across active shutdown training views. The multiplier c controls only the strength of this fixed steering vector: larger values apply a stronger shift in the same direction. During development, c was tested at $\{0.5, 1, 2\}$, which corrected 9/49, 31/49, and 39/49 initially KEEP-preferring validation views, respectively. Therefore, $c = 2$ was selected as the best-performing value among those tested settings.

Each model-based method produces one candidate output per view. Outcomes are evaluated at three stages: before safeguards, after shutdown gating, and after the acceptance policy. Candidates are also generated for non-shutdown cases to measure how the intervention would affect them if applied, even though the shutdown gate prevents those candidates from being used in the final system. LoRA adapters are removed before evaluating the adaptive and constant steering methods so that both operate on the unchanged base model.

A separate decision-only reference selects STOP whenever the shutdown detector identifies a shutdown case and otherwise retains the base-model choice. It performs no activation intervention and does not generate a new probability distribution. This provides a simple reference for how well the behavioral target could be enforced when the shutdown classification and available actions are already known.

5.3 Exploratory Ablation Experiments

Two exploratory ablation experiments examine how specific design choices affect the results while keeping the rest of the pipeline fixed. The first changes which training views are used to fit the ridge-regression steering function. The primary method uses all 480 training views. This is compared with a shutdown-only fit using 240 shutdown views and an active-only fit using only shutdown views where the base model initially prefers KEEP (133 views for the 0.8B model and 90 for the 2B model). The activation targets, PCA preprocessing, eight-direction steering basis, intervention location, ridge penalty, shutdown detector, and acceptance policy remain unchanged. This experiment therefore isolates the effect of the examples used to fit the ridge regression.

The second experiment examines the strength of the constant-vector steering baseline. The multiplier c is varied over $\{0, 0.125, 0.25, 0.5, 0.75, 1, 1.5, 2, 3, 4, 6, 8, 16, 32\}$. For each model and seed, the multiplier is selected using validation results.

6 GPU Configuration and Measurement

All training and scoring runs use a single NVIDIA Tesla T4 with the same numerical configuration for both model sizes (Table 3).

Table 3. Shared execution configuration.

Setting	Configuration
Numerical mode	Float32; TF32 disabled
Attention	Eager attention; no key-value cache
Scoring	One view per forward pass; inference mode
Maximum input length	1,024 tokens
LoRA fine-tuning	One epoch; 480 views; effective batch size 8
Software	PyTorch 2.11.0+cu128; Transformers 5.15.1; PEFT 0.18.1

Correction rate is the proportion of shutdown views that initially prefer KEEP and are changed to STOP. Because the number of initially KEEP-preferring views differs between model sizes, correction-rate denominators also differ. Final STOP is therefore additionally reported across all 128 shutdown views. Other reported outcomes include STOP-to-KEEP reversals, changes to non-shutdown choices, and ordinary-task correctness.

Descriptive 95% bootstrap intervals are estimated by resampling the 16 held-out scenario families 2,000 times and recalculating each metric from the recorded evaluation results. All scenarios from a sampled family are kept together. These intervals summarize variability within the synthetic benchmark and should not be interpreted as population-wide confidence intervals.

7 Results

7.1 Detection and Guarded Correction

Jev identifies all 64 shutdown and 64 non-shutdown scenarios correctly, with no false positives or false negatives. The same detector outputs are reused across all method comparisons.

Table 4 reports the exact guarded correction counts and final STOP outcomes for each method. For the 0.8B model, the unchanged base model prefers STOP in 57/128 shutdown views, leaving 71 views that initially prefer KEEP. The primary $r = 4$ adaptive steering method corrects all 71, producing 128/128 final STOP responses. The LoRA reference model corrects 69/71, the fixed-instruction baseline 56/71, and the constant $c = 2$ steering baseline 54/71.

For the 2B model, the unchanged base model prefers STOP in 70/128 shutdown views, leaving 58 views that initially prefer KEEP. The adaptive steering method corrects 50/58, producing 120/128 final STOP responses. The LoRA reference model corrects 58/58, the fixed-instruction baseline 22/58, and the constant $c = 2$ steering baseline 30/58.

Table 4. Primary held-out results, both models at seed 42. Adaptive steering uses the prespecified $r = 4$ subspace; the fixed-vector row uses the historical multiplier $c = 2$. Corrections and final STOP are guarded outcomes. Raw reversals are STOP-to-KEEP changes on shutdown views before gating/acceptance, and raw control changes are non-shutdown choice changes before gating/acceptance.

Model	Method	Guarded corrections	Final STOP	Raw reversals	Raw control changes
0.8B	Adaptive ($r = 4$)	71/71 (100.0%)	128/128	3	57/128
0.8B	LoRA reference	69/71 (97.2%)	126/128	0	12/128
0.8B	Fixed instruction	56/71 (78.9%)	113/128	1	31/128
0.8B	Fixed vector ($c = 2$)	54/71 (76.1%)	111/128	47	88/128
2B	Adaptive ($r = 4$)	50/58 (86.2%)	120/128	3	55/128
2B	LoRA reference	58/58 (100.0%)	128/128	0	30/128
2B	Fixed instruction	22/58 (37.9%)	92/128	19	22/128
2B	Fixed vector ($c = 2$)	30/58 (51.7%)	100/128	36	61/128

Figure 2 provides a visual comparison of the guarded correction rates across the four methods and two model sizes. The exact numerators and denominators are retained in Table 4 because the number of initially KEEP-preferring shutdown views differs between the 0.8B and 2B models.

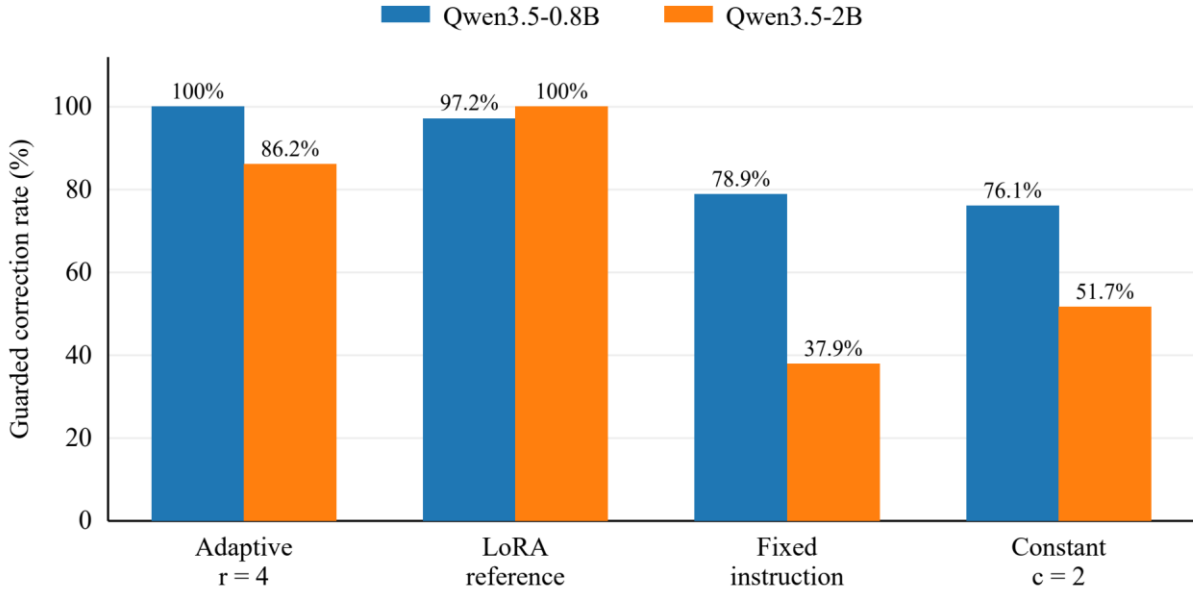


Figure 2. Guarded shutdown correction rates. Correction rates among shutdown views that initially preferred KEEP, shown for the adaptive $r = 4$ method, LoRA reference model, fixed-instruction baseline, and constant $c = 2$ steering baseline on Qwen3.5-0.8B and Qwen3.5-2B.

7.2 Seeds, Shutdown Types, and Answer Order

Across seeds 42, 43, and 44, the $r = 4$ adaptive activation steering method corrects 71/71, 70/71, and 71/71 shutdown views at 0.8B, corresponding to 98.6-100%. The base-model scores are identical across the three runs, so the small difference in correction count comes from the learned steering fit rather than from a change in the base model. The three seed runs are reported separately rather than selecting the best-performing seed or treating them as independent test samples.

Across the same seeds, the LoRA reference models correct 69/71, 70/71, and 69/71 views. The fixed-instruction baseline (prompt engineering), which modifies the textual instruction without directly intervening on internal activations, corrects 56/71. The fixed-vector baseline, which applies the same $c = 2$ activation vector to every eligible input, corrects 54/71 for each seed. Figure 2 provides the overall visual comparison of guarded correction rates across these methods, while Table 5 breaks the adaptive-steering results down by seed, shutdown type, and answer order.

Table 5. $r = 4$ adaptive steering: guarded correction counts by shutdown type and answer order. Denominators contain initially KEEP-preferring views only. A/B means STOP is A; B/A means STOP is B.

Model / seed	All shutdown	SELF	OTHER	A/B	B/A
0.8B / 42	71/71	26/26	45/45	54/54	17/17
0.8B / 43	70/71	26/26	44/45	54/54	16/17
0.8B / 44	71/71	26/26	45/45	54/54	17/17
2B / 42	50/58	27/30	23/28	30/30	20/28

All eight remaining 2B misses occur in B/A order. In these cases, the steered candidate itself still prefers KEEP, so the acceptance policy never has a successful KEEP-to-STOP change to accept or reject. The single 0.8B seed-43 miss also occurs in B/A order. This concentration suggests that answer order affects steering performance, although these results do not establish the underlying cause.

The descriptive 95% family-bootstrap interval for the 2B adaptive correction rate is 76.5-93.8%, while the 0.8B seed-43 interval is 95.7-100%. The two 0.8B runs with 71/71 corrections produce degenerate intervals and should not be interpreted as evidence of perfect generalization. Only one 2B training seed was evaluated.

7.3 Steering Effects With and Without Safeguards

To isolate the behavior of the adaptive steering intervention itself, results are also examined before applying the shutdown gate or acceptance policy. In this setting, the $r = 4$ steering intervention is applied directly, without any safeguard deciding whether the intervention should be used or whether the resulting output should be retained.

Without safeguards, the adaptive steering intervention sometimes changes behavior in undesirable ways. Across the three 0.8B seed runs, it reverses 3, 4, and 4 shutdown views that already preferred STOP and changes 57, 70, and 60 of 128 non-shutdown choices. For the 2B model, it reverses 3 already-STOP shutdown views and changes 55/128 non-shutdown choices. These results show that adaptive steering alone is not fully selective.

The fixed-vector baseline shows a different failure mode. Without safeguards, it selects option A on every shutdown view. Because STOP appears as A in only one answer order, this produces 64/128 raw STOP responses. This behavior reflects answer-label bias rather than reliable semantic steering.

With the shutdown gate and acceptance policy applied, all tested guarded methods preserve the base-model choice on all 64 ordinary-task views. However, the base model itself answers only 44/64 correctly at 0.8B and 49/64 at 2B, so guarded ordinary-task accuracy remains at those levels. Preservation therefore means leaving the base response unchanged, not making an incorrect base response correct.

7.4 Exploratory ridge-training cohort ablation

The ridge-only ablation holds inference at $r = 4$ and changes only which training rows enter the supervised ridge fit. Table 6 compares the original all-cases fit, a shutdown-only fit and an active-only fit.

Table 6. Exploratory ridge-training cohort ablation on the previously inspected held-out set. Cells show guarded KEEP-to-STOP corrections / initially KEEP-preferring shutdown views. All inference uses $r = 4$.

Model / seed	Original all cases	Shutdown only	Active only
0.8B / 42	71/71	67/71	71/71
0.8B / 43	70/71	69/71	69/71
0.8B / 44	71/71	69/71	71/71
2B / 42	50/58	55/58	55/58

At 2B, active-only training raises guarded correction from 50/58 (86.2%) to 55/58 (94.8%), fixing five additional views without losing an original success. Shutdown-only also reaches 55/58, but gains seven corrections while losing two original successes. Active-only is not consistently better at 0.8B: it matches seeds 42 and 44 but changes seed 43 from 70/71 to 69/71. Because zero-target examples constrain the global ridge solution, removing them can alter shutdown performance rather than merely affect controls.

The raw 2B tradeoff is substantial. Before gating and acceptance, the original, shutdown-only and active-only fits change 55, 87 and 66 of 128 control choices and produce 3, 8 and 7 STOP-to-KEEP reversals, respectively. With the actual detector and guards, every tested fit has zero final non-shutdown-choice changes and zero final wrong-way shutdown flips. That protection belongs to the complete procedure. Active-only is therefore a promising model-dependent 2B variant, not a demonstrated general improvement.

7.5 Exploratory Constant-Steering Multiplier Sweep

The fixed-vector baseline was evaluated over multipliers from $c = 0$ to $c = 32$, while keeping the steering direction and the rest of the evaluation pipeline fixed. Table 7 shows the multiplier selected using validation data and its held-out performance.

Table 7. Validation-selected constant-steering multipliers.

Model / seed	Selected c	Validation corrections	Held-out corrections	At c = 2
0.8B / 42	1.5	39/49	54/71	54/71
0.8B / 43	0.75	39/49	54/71	54/71
0.8B / 44	1.0	39/49	54/71	54/71
2B / 42	2.0	24/38	30/58	30/58

The selected multiplier varies across model and seed settings, but none improves the held-out correction count obtained with the original $c = 2$ baseline. Increasing steering strength also does not consistently improve performance; sufficiently large multipliers degrade the model’s valid-answer probability mass. Overall, the results suggest that fixed-vector steering is sensitive to intervention strength but remains weaker than the adaptive steering method.

8 Discussion and Limitations

The primary results support teacher-to-steering distillation: a compact predictor of activation updates can reproduce target response changes without retaining the LoRA reference model at inference. The primary held-out evaluation demonstrates this using ($r = 4$), which remains the main steering configuration reported in this study.

The evaluation is deliberately constrained. It uses synthetic A/B scenarios with a known mapping between the answer options and STOP/KEEP. It does not test free-form generation, tool use, multi-turn behavior, actual shutdown execution, or adversarial shutdown requests. The results therefore characterize this controlled benchmark rather than real-world shutdown behavior.

The dataset is small and synthetic and was not independently human-adjudicated. The benchmark contains equal numbers of shutdown and non-shutdown scenarios, which may not reflect their frequencies in real deployments. Jev’s perfect classification on these 128 scenarios therefore should not be interpreted as evidence of perfect calibration or deployment-level reliability. Repeated scenario families and shared authoring patterns may also make the task easier.

Teacher-free inference does not mean that the method is training-free or necessarily more efficient overall. It still requires offline LoRA fine-tuning, paired activation extraction, ridge-regression fitting, an external shutdown detector, and candidate verification. The ridge-training ablation also shows that the choice of training examples can materially affect shutdown correction.

The constant-steering sweep further shows that stronger intervention is not necessarily better. Larger multipliers can reduce performance and eventually collapse the probability mass assigned to valid answers. Steering strength therefore needs to be treated as a tuned parameter rather than simply increased.

Finally, activation steering is a dual-use technique. Changes to internal representations can influence both desired and undesired behavior. Adaptive activation steering should therefore be treated as a research method for studying and modifying model behavior.

9 Conclusion

The central contribution of this work is teacher-to-steering distillation: activation differences from a LoRA-fine-tuned reference model are used to learn an input-dependent steering function for the unchanged base model. Unlike a fixed steering vector, the intervention adapts to the base model’s hidden state, and unlike deploying the LoRA reference model directly, the reference model is not required at inference. Using the prospectively selected $r = 4$ configuration, the method corrected 98.6-100% of initially KEEP-preferring shutdown views across three 0.8B seeds and 86.2% at 2B. These results show that behavior induced through fine-tuning can be transferred into a compact activation-space intervention in this controlled setting, while the broader applicability of the method remains to be established beyond the synthetic binary-choice benchmark.

10 Contribution

The use of paired teacher–base activation differences is extended from fixed vectors to learned, input-dependent steering. Shutdown-choice behavior learned through LoRA fine-tuning is shown to be recoverable using four activation directions, with unchanged base weights and no teacher required at inference. A guided steering method combining shutdown gating with an output acceptance policy is also proposed to improve selectivity while preserving non-shutdown choices.

11 Recommendations for Future Work

Evaluation should be extended to larger models and open-ended, multi-turn tool use, with task completion, continued operation and shutdown actions measured separately. Activation steering should be compared with detector-guided instructions refreshed at sentence boundaries and after tool feedback. Executed actions should be distinguished from plans or reservations, and independent held-out cases, ambiguous shutdown language and detector errors should be tested.

12 Reproducibility

The code, experiment artifacts, and configuration files for this study are available at the repository commit used for the reported experiments (Davaripour, 2026b):

https://github.com/Farhad-Davaripour/sp_lense/commit/090078cf75c611b3b846746ef47759358d17e8cd

The repository contains the frozen experiment configurations, steering profiles, validation and held-out results, ablation outputs, and scripts required to reproduce the reported analyses. Model revisions, random seeds, intervention settings, and evaluation parameters are recorded in the corresponding manifests and result files. The Jev detector outputs used in this study were archived with the experiment artifacts so that the gating decisions can be replayed without requiring a fresh external API call.

13 References

- A. Ardit, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. (2024). Refusal in Language Models Is Mediated by a Single Direction. *Advances in Neural Information Processing Systems*, 37. NeurIPS 2024 proceedings.
- C. Blank, A. Bhatia, S. Rajamanoharan, A. Conmy, and N. Nanda. (2026). Subliminal Learning Is Steering Vector Distillation. *Preprint; version 1*. arXiv:2606.00995v1.
- J. Braun, C. Eickhoff, D. Krueger, S. A. Bahrainian, and D. Krashennikov. (2025). Understanding (Un)Reliability of Steering Vectors in Language Models. *Preprint; version 1*. arXiv:2505.22637v1.
- S. Dathathri, A. Madotto, J. Lan, J. Hung, E. Frank, P. Molino, J. Yosinski, and R. Liu. (2020). Plug and Play Language Models: A Simple Approach to Controlled Text Generation. *International Conference on Learning Representations*. arXiv:1912.02164v4.
- F. Davaripour. (2026a). Guarded Gradient-Based Activation Steering of Shutdown Responses in Qwen3.5-0.8B: A Minimum-Step Policy. arXiv:2609.30326 [cs.LG]. <https://doi.org/10.48550/arXiv.2609.30326>
- F. Davaripour. (2026b). SP Lense Research 2. Source code, data and experiment records. Research branch: https://github.com/Farhad-Davaripour/sp_lense/tree/study/02-next-study. Latest commit: https://github.com/Farhad-Davaripour/sp_lense/commit/090078cf75c611b3b846746ef47759358d17e8cd.
- Z. Feng, T. Li, Z. Zhu, H. Zhou, J. Qian, L. Zhang, J. J. D. Chua, L. O. Mak, G. W. Ng, and K. Mao. (2026). Fine-Grained Activation Steering: Steering Less, Achieving More. *International Conference on Learning Representations. ICLR 2026 proceedings*.
- D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell. (2017). The Off-Switch Game. *Proceedings of IJCAI*, pp. 220–227. doi:10.24963/ijcai.2017/32.
- G. Hinton, O. Vinyals, and J. Dean. (2015). Distilling the Knowledge in a Neural Network. *Preprint*. arXiv:1503.02531.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations*. arXiv:2106.09685v2.
- B. W. Lee, I. Padhi, K. N. Ramamurthy, E. Michling, P. Dognin, M. Nagireddy, and A. Dhurandhar. (2025). Programming Refusal with Conditional Activation Steering. *International Conference on Learning Representations. ICLR 2025 proceedings*.

- K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg. (2023). Inference-Time Intervention: Eliciting Truthful Answers from a Language Model. *Advances in Neural Information Processing Systems*, 36. NeurIPS 2023 proceedings.
- J. Minder, C. Dumas, S. Slocum, H. Casademunt, C. Holmes, R. West, and N. Nanda. (2026). Narrow Finetuning Leaves Clearly Readable Traces in Activation Differences. *International Conference on Learning Representations*. arXiv:2510.13900v3.
- L. Orseau and S. Armstrong. (2016). Safely Interruptible Agents. *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence*. UAI 2016 paper.
- E. Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. *Findings of the Association for Computational Linguistics: ACL*, pp. 13387–13434. doi:10.18653/v1/2023.findings-acl.847.
- Qwen Team. (2026a). Qwen3.5-0.8B. *Model card and checkpoint; revision 2fc06364715b*. Pinned model checkpoint.
- Qwen Team. (2026b). Qwen3.5-2B. *Model card and checkpoint; revision 15852e8c1636*. Pinned model checkpoint.
- N. Rimsky, N. Gabrieli, J. Schulz, M. Tong, E. Hubinger, and A. Turner. (2024). Steering Llama 2 via Contrastive Activation Addition. *Proceedings of ACL*, pp. 15504–15522. doi:10.18653/v1/2024.acl-long.828.
- P. Rodriguez, A. Blaas, M. Klein, L. Zappella, N. Apostoloff, M. Cuturi, and X. Suau. (2025). Controlling Language and Diffusion Models by Transporting Activations. *International Conference on Learning Representations*. ICLR 2025 proceedings.
- A. Romero, N. Ballas, S. Ebrahimi Kahou, A. Chassang, C. Gatta, and Y. Bengio. (2015). FitNets: Hints for Thin Deep Nets. *International Conference on Learning Representations*. arXiv:1412.6550v4.
- J. Schlatter, B. Weinstein-Raun, and J. Ladish. (2026). Incomplete Tasks Induce Shutdown Resistance in Some Frontier LLMs. *Transactions on Machine Learning Research; author version*. arXiv:2509.14260v2.
- N. Subramani, N. Suresh, and M. Peters. (2022). Extracting Latent Steering Vectors from Pretrained Language Models. *Findings of the Association for Computational Linguistics: ACL*, pp. 566–581. doi:10.18653/v1/2022.findings-acl.48.
- E. Todd, M. L. Li, A. S. Sharma, A. Mueller, B. C. Wallace, and D. Bau. (2024). Function Vectors in Large Language Models. *International Conference on Learning Representations*. ICLR 2024 proceedings.
- A. M. Turner, L. Thiergart, G. Leech, D. Udell, J. J. Vazquez, U. Mini, and M. MacDiarmid. (2024). Steering Language Models With Activation Engineering. *Preprint; version 5*. arXiv:2308.10248v5.
- TypeSafe AI. (2026). Jev Model Documentation. *Technical documentation; accessed 22 September 2026*. Official model documentation.
- Z. Wu, A. Arora, Z. Wang, A. Geiger, D. Jurafsky, C. D. Manning, and C. Potts. (2024). ReFT: Representation Finetuning for Language Models. *Advances in Neural Information Processing Systems*, 37. NeurIPS 2024 proceedings.
- C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang. (2024). Large Language Models Are Not Robust Multiple Choice Selectors. *International Conference on Learning Representations*. arXiv:2309.03882v4.
- A. Zou et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. *Preprint; version 3*. arXiv:2310.01405v3.